

High Performance Computing

Name: Thulasi Shylasri

HTNO: 2303A51876

Batch-14

Week-11

LAB Assignment-11 (18/03/2026)

Assignment 1: Basic Point-to-Point Communication using MPI Send/Receive

Scenario

A distributed logging system needs to transfer status messages from a master process to a worker process for monitoring.

Objective

Understand basic point-to-point communication using MPI Send and Receive in Python.

Tasks

1. Write an MPI program with two processes.
2. Use process with rank 0 to send a message string to process with rank 1.
3. Receive and display the message at the receiver process.
4. Verify correct execution using multiple runs.

Learning Outcomes

- Basic MPI program structure
- Point-to-point communication
- Role of ranks in MPI

```

(Reading database ... 58908 files and directories currently installed.)
Preparing to unpack .../6-libxpat1-dev_2.6.1-2ubuntu0.4_amd64.deb ...
Unpacking libxpat1-dev:amd64 (2.6.1-2ubuntu0.4) ...
Selecting previously unselected package libjs-underscore.
Preparing to unpack .../71-libjs-underscore_1.13.4-dfsg+~1.11.4-3_all.deb ...
Unpacking libjs-underscore (1.13.4-dfsg+~1.11.4-3) ...
Selecting previously unselected package libjs-sphinxdoc.
Preparing to unpack .../72-libjs-sphinxdoc_7.2.6-6_all.deb ...
Unpacking libjs-sphinxdoc (7.2.6-6) ...
Selecting previously unselected package libpython3.12-dev:amd64.
Preparing to unpack .../3-libpython3.12-dev_3.12.3-1ubuntu0.12_amd64.deb ...
Unpacking libpython3.12-dev:amd64 (3.12.3-1ubuntu0.12) ...
Selecting previously unselected package libpython3-dev:amd64.
Preparing to unpack .../4-libpython3-dev_3.12.3-1ubuntu0.12_amd64.deb ...
Unpacking libpython3-dev:amd64 (3.12.3-1ubuntu0.12) ...
Selecting previously unselected package python3.12-dev.
Preparing to unpack .../5-python3.12-dev_3.12.3-1ubuntu0.12_amd64.deb ...
Unpacking python3.12-dev (3.12.3-1ubuntu0.12) ...
Selecting previously unselected package python3-dev.
Preparing to unpack .../6-python3-dev_3.12.3-1ubuntu0.12_amd64.deb ...
Unpacking python3-dev (3.12.3-1ubuntu0.12) ...
Selecting previously unselected package python3-wheel.
Preparing to unpack .../7-python3-wheel_0.42.0-2_all.deb ...
Unpacking python3-wheel (0.42.0-2) ...
Selecting previously unselected package python3-pip.
Preparing to unpack .../8-python3-pip_24.0+dfsg-1ubuntu1.3_all.deb ...
Unpacking python3-pip (24.0+dfsg-1ubuntu1.3) ...
Setting up python3-wheel (0.42.0-2) ...
Setting up libxpat1-dev:amd64 (2.6.1-2ubuntu0.4) ...
Setting up python3-pip (24.0+dfsg-1ubuntu1.3) ...
Setting up libjs-underscore (1.13.4-dfsg+~1.11.4-3) ...
Setting up libpython3.12-dev:amd64 (3.12.3-1ubuntu0.12) ...
Setting up python3.12-dev (3.12.3-1ubuntu0.12) ...
Setting up libjs-sphinxdoc (7.2.6-6) ...
Setting up libpython3-dev:amd64 (3.12.3-1ubuntu0.12) ...
Setting up python3-dev (3.12.3-1ubuntu0.12) ...
Processing triggers for man-db (2.12.0-4build2) ...
neerati@neerati-laptop:~$ sudo apt install python3-mpi4py
python3-mpi4py (0.1.6)
Report bugs to http://www.open-mpi.org/community/help/

```

```

GNU nano 7.2 mpi_send_receive.py *
from mpi4py import MPI

# Initialize MPI
comm = MPI.COMM_WORLD
rank = comm.Get_rank()

if rank == 0:
    # Sender (Master)
    message = "Hello from Master Process (Rank 0)"
    comm.send(message, dest=1)
    print("Message sent from Rank 0")

elif rank == 1:
    # Receiver (Worker)
    received_message = comm.recv(source=0)
    print("Message received at Rank 1:")
    print(received_message)

```

```

See /usr/share/doc/python3.12/README.venv for more information.

WARNING: If you believe this is a mistake, please contact your Python installation or OS distribution provider. You can override this, at the risk of breaking
your Python installation or OS, by passing --break-system-packages.
hint: See PEP 668 for the detailed specification.
neerati@neerati-laptop:~$ sudo apt install python3-mpi4py
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
The following packages were automatically installed and are no longer required:
  libllvm10 libxslt1-dev
Use 'sudo apt autoremove' to remove them.
The following additional packages will be installed:
  mpi-default-bin
Suggested packages:
  python3-numpy
The following NEW packages will be installed:
  mpi-default-bin python3-mpi4py
0 upgraded, 2 newly installed, 0 to remove and 0 not upgraded.
Need to get 468 kB of archives.
After this operation, 1935 kB of additional disk space will be used.
Get:1 http://archive.ubuntu.com/ubuntu noble/universe amd64 mpi-default-bin amd64 1.15build1 [2376 B]
Get:2 http://archive.ubuntu.com/ubuntu noble/universe amd64 python3-mpi4py amd64 3.1.5-5ubuntu2 [466 kB]
Fetched 468 kB in 2s (218 kB/s)
Selecting previously unselected package mpi-default-bin.
(Reading database ... 51982 files and directories currently installed.)
Preparing to unpack .../mpi-default-bin_1.15build1_amd64.deb ...
Unpacking mpi-default-bin (1.15build1) ...
Selecting previously unselected package python3-mpi4py.
Preparing to unpack .../python3-mpi4py_3.1.5-5ubuntu2_amd64.deb ...
Unpacking python3-mpi4py (3.1.5-5ubuntu2) ...
Setting up mpi-default-bin (1.15build1) ...
Setting up python3-mpi4py (3.1.5-5ubuntu2) ...
neerati@neerati-laptop:~$ python3 -c "from mpi4py import MPI; print('mpi4py installed!)"
mpi4py installed
neerati@neerati-laptop:~$ mpiexec -n 2 python3 mpi_send_receive.py
Message sent from Rank 0
Message received at Rank 1:
Hello from Master Process (Rank 0)
neerati@neerati-laptop:~$

```

```
GNU nano 7.2 mpi_send_receive.py
from mpi4py import MPI

comm = MPI.COMM_WORLD
rank = comm.Get_rank()

print(f"I am process {rank}")

if rank == 0:
    message = "Hello from Master Process (Rank 0)"
    comm.send(message, dest=1)

elif rank == 1:
    received_message = comm.recv(source=0)
    print("Received:", received_message)

serati_soumya@LAPTOP-ETNJBQA1:~$ nano mpi_send_receive.py
serati_soumya@LAPTOP-ETNJBQA1:~$ mpiexec -n 2 python3 mpi_send_receive.py
I am process 0
I am process 1
Received: Hello from Master Process (Rank 0)
serati_soumya@LAPTOP-ETNJBQA1:~$
```

Key Points:

The program demonstrated basic point-to-point communication using MPI by creating two processes with `mpiexec -n 2`, where each process was assigned a unique rank. The process with rank 0 acted as the sender (master) and transmitted a message, while the process with rank 1 acted as the receiver (worker) and successfully received the message. Communication was carried out using `comm.send()` and `comm.recv()` functions. The message transfer was accurate and consistent across multiple executions, confirming reliable communication.

Observations

The program was executed using two processes. It was observed that the process with rank 0 successfully sent a message to the process with rank 1. The receiving process displayed the message correctly on the screen. The output remained consistent across multiple executions, which confirms that the communication between processes was working properly.

Explanation

In this program, MPI assigns a unique rank to each process. The process with rank 0 acts as the sender, while the process with rank 1 acts as the receiver. The sender uses the `send()` function to transmit a message, and the receiver

uses the `recv()` function to accept it. This demonstrates basic point-to-point communication in MPI, where data is transferred directly between two processes.

Conclusion

The experiment successfully demonstrated simple message passing between two processes using MPI. It helped in understanding the concept of ranks and basic communication functions like send and receive.

Assignment 2: Integer Data Exchange between Multiple MPI Processes

Scenario

A parallel sensor network distributes integer readings from a control node to compute nodes.

Objective

Use MPI Send and Receive for integer data communication among multiple processes.

Tasks

1. Run the MPI program with at least 4 processes.
2. Let process rank 0 send different integer values to ranks 1, 2, and 3.
3. Each receiving process prints the received value along with its rank.
4. Ensure correct use of tags to differentiate messages.

Learning Outcomes

- Message tagging
- Multiple send/receive operations
- Process coordination in MPI

```

[GNU nano 2.9.3] mpi_integer_exchange.py
from mpi4py import MPI

comm = MPI.COMM_WORLD
rank = comm.Get_rank()
size = comm.Get_size()

# Unsure at least 4 processes
if size < 4:
    if rank == 0:
        print("Please run with at least 4 processes")
        exit()

# Master process
if rank == 0:
    data = [10, 20, 30] # 3 integers to send
    comm.send(data[0], dest=1, tag=101)
    comm.send(data[1], dest=2, tag=102)
    comm.send(data[2], dest=3, tag=103)

    print("Master sent values:", data)

# Worker processes
else:
    received_data = comm.recv(source=0, tag=100 + rank)
    print(f"Process {rank} received value: {received_data}")

```

```

seerat1_soumya@APTOP-ETHJ8QQA:~$ nano mpi_integer_exchange.py
seerat1_soumya@APTOP-ETHJ8QQA:~$ nano mpi_integer_exchange.py
seerat1_soumya@APTOP-ETHJ8QQA:~$ mpispec -n 4 python3 mpi_integer_exchange.py
Master sent values: [10, 20, 30]
Process 1 received value: 10
Process 2 received value: 20
Process 3 received value: 30
seerat1_soumya@APTOP-ETHJ8QQA:~$ mpispec -n 4 python3 mpi_integer_exchange.py
Process 1 received value: 10
Process 2 received value: 20
Master sent values: [10, 20, 30]
Process 3 received value: 30
seerat1_soumya@APTOP-ETHJ8QQA:~$ mpispec -n 4 python3 mpi_integer_exchange.py
Master sent values: [10, 20, 30]
Process 1 received value: 10
Process 2 received value: 20
Process 3 received value: 30
seerat1_soumya@APTOP-ETHJ8QQA:~$

```

Observations

The program was executed with four processes. It was observed that the master process (rank 0) sent different integer values to processes 1, 2, and 3. Each process received its respective value correctly and printed it along with its rank. Although the order of output changed in different runs, the data received by each process was always accurate.

Explanation

In this program, the master process sends multiple messages using different tags. Each worker process receives the message based on its assigned tag. Tags play an important role in distinguishing between different messages, ensuring that each process gets the correct data even when multiple communications happen simultaneously.

Conclusion

This assignment helped in understanding how multiple processes can communicate efficiently using MPI. It also showed the importance of message tagging in handling multiple communications correctly.

Assignment 3: One-to-One Communication Pattern using MPI

Scenario

In a distributed simulation, each process must exchange boundary data with its neighboring process.

Objective

Implement pairwise Send/Receive communication among MPI processes.

Tasks

1. Run the program with an even number of processes.
2. Each process with even rank sends data to the next odd-ranked process.
3. Odd-ranked processes receive and print the data.
4. Analyze blocking behavior of Send/Receive.

Learning Outcomes

- Pairwise communication patterns
- Blocking communication behavior
- Process synchronization



```
python3 mpi_hello.py
from mpi4py import MPI

comm = MPI.COMM_WORLD
rank = comm.Get_rank()
size = comm.Get_size()

# Check if number of processes is even
if size % 2 != 0:
    if rank == 0:
        print("Please run with even number of processes")
    exit()

# Send data from even processes
if rank % 2 == 0:
    message = f"Hello from process {rank}"
    comm.Send(message, dest=rank + 1)
    print(f"Process {rank} sent message to {rank + 1}")

# All odd processes receive
else:
    received_message = comm.Recv(source=rank - 1)
    print(f"Process {rank} received {received_message}")

print(f"I am process {rank}")
```



```

sccrati_soumya@LAPTOP-CTHJHQA:-$ nano mpi_pairwise.py
sccrati_soumya@LAPTOP-CTHJHQA:-$ mpiexec -n 4 python3 mpi_pairwise.py
Process 0 sent message to 1
I am process 0
Process 1 received: Hello from process 0
I am process 1
Process 1 sent message to 3
I am process 2
Process 3 received: Hello from process 2
I am process 3
sccrati_soumya@LAPTOP-CTHJHQA:-$ mpiexec -n 2 python3 mpi_pairwise.py
Process 1 received: Hello from process 0
I am process 1
Process 0 sent message to 1
I am process 0
sccrati_soumya@LAPTOP-CTHJHQA:-$ mpiexec -n 8 python3 mpi_pairwise.py
Process 1 sent message to 3
I am process 2
Process 4 sent message to 5
I am process 4
Process 0 sent message to 1
I am process 0
Process 1 received: Hello from process 0
I am process 1
Process 3 received: Hello from process 2
I am process 3
Process 5 received: Hello from process 4
I am process 5
sccrati_soumya@LAPTOP-CTHJHQA:-$ |

```

Observations

The program was run with an even number of processes. It was observed that even-ranked processes successfully sent messages to the next odd-ranked processes. The odd-ranked processes received the messages and displayed them correctly. The execution order varied due to parallel processing, but communication remained accurate.

Explanation

In this communication pattern, processes are divided into pairs. Even-ranked processes act as senders, and odd-ranked processes act as receivers. The use of blocking communication ensures that the sender waits until the receiver is ready, which helps maintain synchronization between processes.

Conclusion

The assignment successfully demonstrated pairwise communication between processes. It also provided a clear understanding of how blocking communication ensures proper synchronization.

Assignment 4: Deadlock Analysis in MPI Send/Receive

Scenario

A parallel application experiences a hang due to improper ordering of communication calls.

Objective

Understand deadlock conditions in blocking MPI Send and Receive.

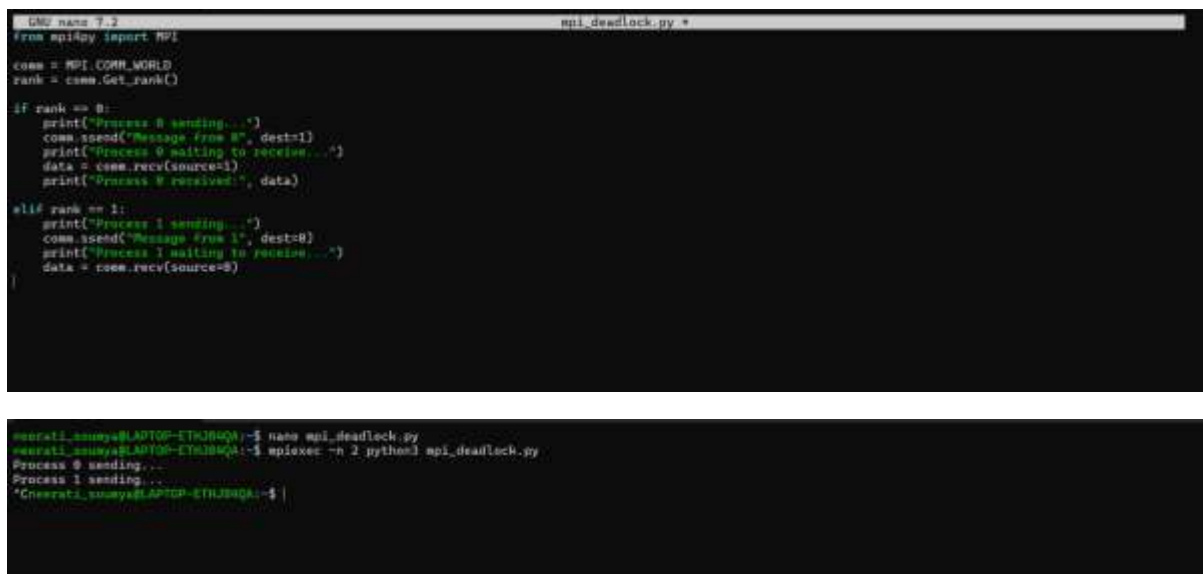
Tasks

1. Write an MPI program where two processes call send() first and then recv().
2. Observe and explain the deadlock behavior.
3. Modify the program to avoid deadlock by reordering communication.
4. Document the corrected communication sequence.

Learning Outcomes

- Deadlock in MPI
- Blocking communication risks
- Correct ordering of Send/Receive

Deadlock:



```
GNU nano 2.2 mpi_deadlock.py *
from mpi4py import MPI

comm = MPI.COMM_WORLD
rank = comm.Get_rank()

if rank == 0:
    print("Process 0 sending...")
    comm.send("Message from 0", dest=1)
    print("Process 0 waiting to receive...")
    data = comm.recv(source=1)
    print("Process 0 received:", data)

elif rank == 1:
    print("Process 1 sending...")
    comm.send("Message from 1", dest=0)
    print("Process 1 waiting to receive...")
    data = comm.recv(source=0)

```

```
venkatesh@LAPTOP-ETHJ8QQA:~$ nano mpi_deadlock.py
venkatesh@LAPTOP-ETHJ8QQA:~$ mpirun -n 2 python3 mpi_deadlock.py
Process 0 sending...
Process 1 sending...
^Cvenkatesh@LAPTOP-ETHJ8QQA:~$
```


Avoiding Deadlock:

```
GNU nano 2.2 mpi_deadlock.py *
from mpi4py import MPI

comm = MPI.COMM_WORLD
rank = comm.Get_rank()

if rank == 0:
    comm.send("Message from 0", dest=1)
    data = comm.recv(source=1)
    print("Process 0 received:", data)

elif rank == 1:
    data = comm.recv(source=0)
    print("Process 1 received:", data)
    comm.send("Message from 1", dest=0)
```

```
seerat1_sunny@LAPTOP-ETHJ8QAQ:~$ nano mpi_deadlock.py
seerat1_sunny@LAPTOP-ETHJ8QAQ:~$ mpiexec -n 2 python3 mpi_deadlock.py
Process 0 sending...
Process 1 sending...
seerat1_sunny@LAPTOP-ETHJ8QAQ:~$ nano mpi_deadlock.py
seerat1_sunny@LAPTOP-ETHJ8QAQ:~$ mpiexec -n 2 python3 mpi_deadlock.py
Process 0 received: Message from 1
Process 1 received: Message from 0
seerat1_sunny@LAPTOP-ETHJ8QAQ:~$
```

Observations

When the program was executed using synchronous send, both processes attempted to send messages at the same time before receiving. As a result, the program got stuck and did not proceed further. It had to be manually stopped. After modifying the program by changing the order of send and receive operations, the program executed successfully without any issues.

Explanation

Deadlock occurs when two or more processes wait indefinitely for each other to complete their operations. In this case, both processes were waiting for the other to receive the message, causing the program to hang. By reordering the communication (making one process receive first), the deadlock was avoided and proper execution was achieved.

Conclusion

This assignment clearly demonstrated how improper ordering of communication can lead to deadlock. It also showed how correct sequencing.